

Farm Market Platform

Professional Project Documentation

Version: 1.0.0
Last Updated: April 2026
Document Type: Technical Overview & System Architecture

Table of Contents

- 1. [Executive Summary](#)
 - 2. [Project Overview](#)
 - 3. [System Architecture](#)
 - 4. [Technology Stack](#)
 - 5. [Core Features](#)
 - 6. [User Roles & Permissions](#)
 - 7. [Data Models](#)
 - 8. [Authentication & Security](#)
 - 9. [API Architecture](#)
 - 10. [Frontend Architecture](#)
 - 11. [Progressive Web App \(PWA\)](#)
 - 12. [Deployment & Infrastructure](#)
 - 13. [Future Enhancements](#)
-

Executive Summary

Farm Market Platform is a modern web-based marketplace designed to connect farmers directly with buyers in the Tamale region of Ghana. The platform eliminates intermediaries, enabling farmers to list their produce at competitive prices while providing buyers with direct access to fresh agricultural products.

Key Highlights

- **Target Market:** Agricultural communities in Northern Ghana, specifically Tamale region
 - **Primary Users:** Farmers, Buyers, and Platform Administrators
 - **Core Value Proposition:** Direct farmer-to-buyer connections, transparent pricing, and mobile-first accessibility
 - **Technology Approach:** Progressive Web App (PWA) for offline capability and mobile optimization
 - **Deployment:** Cloud-based with scalable architecture
-

Project Overview

Problem Statement

Traditional agricultural markets in Ghana face several challenges:

- Farmers lack direct access to buyers, relying on middlemen who reduce profit margins
- Price information is opaque and inconsistent across markets
- Buyers have limited visibility into available produce and pricing
- Communication barriers between farmers and buyers
- No centralized platform for market price discovery

Solution

Farm Market Platform addresses these challenges by providing:

- **Direct Marketplace:** Farmers can list produce with photos, quantities, and prices
- **Transparent Pricing:** Real-time market price information for major crops
- **Mobile Accessibility:** Progressive Web App works on any device, including low-end smartphones
- **Offline Capability:** Core features work without internet connectivity
- **Direct Communication:** Built-in phone and WhatsApp integration for instant contact
- **Admin Dashboard:** Platform management tools for monitoring and price updates

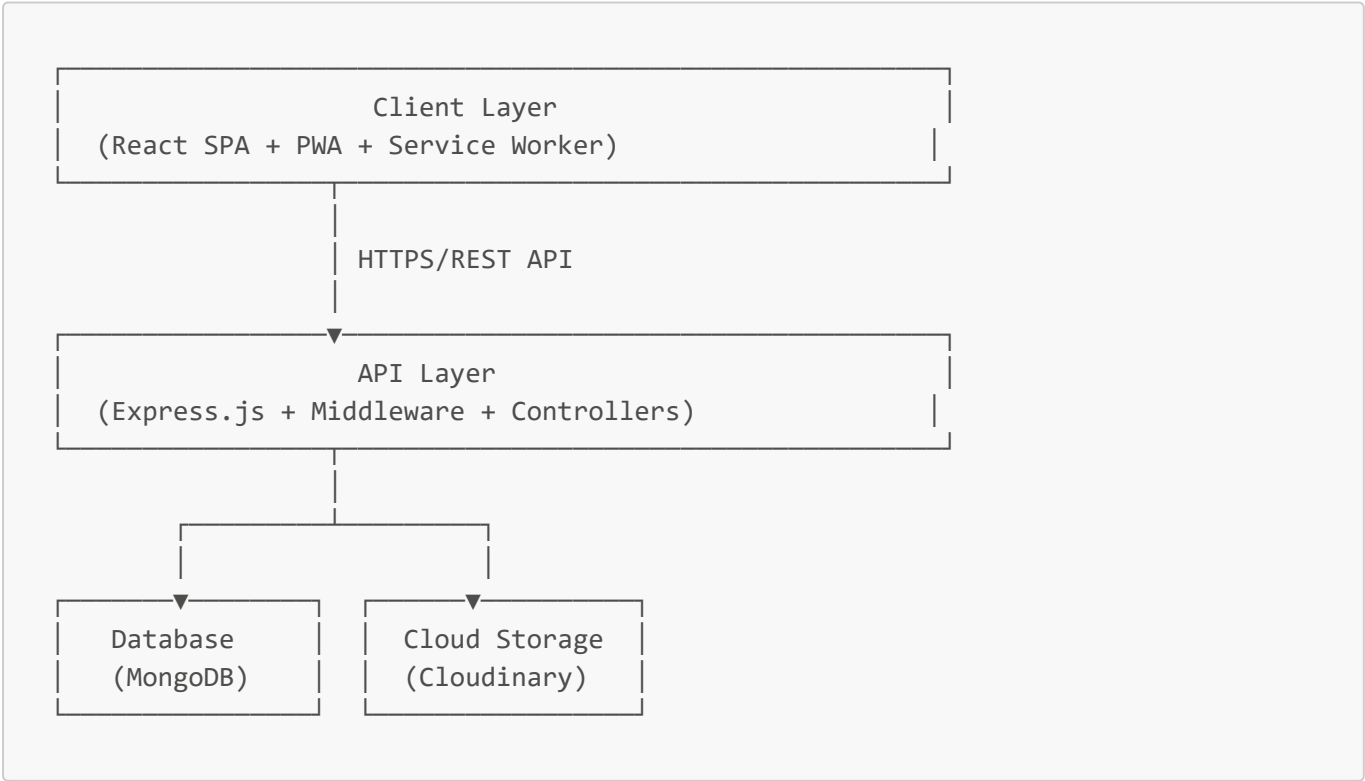
Target Audience

1. **Farmers:** Small to medium-scale farmers growing maize, rice, and yam
2. **Buyers:** Wholesalers, retailers, food processors, and individual consumers
3. **Administrators:** Platform managers responsible for oversight and market price updates

System Architecture

High-Level Architecture

The platform follows a modern **client-server architecture** with clear separation of concerns:



Architecture Principles

1. **Separation of Concerns:** Clear boundaries between presentation, business logic, and data layers
2. **RESTful Design:** Standard HTTP methods and resource-based URLs
3. **Stateless API:** JWT-based authentication for scalability
4. **Mobile-First:** Responsive design optimized for mobile devices
5. **Offline-First:** Service Worker enables offline functionality
6. **Security by Design:** Authentication, authorization, and input validation at every layer

Technology Stack

Frontend Technologies

Technology	Version	Purpose
React	18.3.1	UI framework for building interactive interfaces
TypeScript	5.8.3	Type-safe JavaScript for better code quality
Vite	5.4.19	Fast build tool and development server
React Router	6.30.1	Client-side routing and navigation
TanStack Query	5.83.0	Server state management and caching
Tailwind CSS	3.4.17	Utility-first CSS framework
shadcn/ui	Latest	Accessible component library built on Radix UI
Radix UI	Latest	Unstyled, accessible UI primitives
React Hook Form	7.61.1	Form state management and validation
Zod	3.25.76	Schema validation for forms and data
Lucide React	0.462.0	Icon library
Recharts	2.15.4	Data visualization for analytics
date-fns	3.6.0	Date manipulation and formatting

Backend Technologies

Technology	Version	Purpose
Node.js	Latest LTS	JavaScript runtime environment
Express.js	4.21.2	Web application framework
MongoDB	8.18.0	NoSQL database for flexible data storage
Mongoose	8.18.0	MongoDB object modeling and validation
JWT	9.0.2	JSON Web Tokens for authentication
bcryptjs	2.4.3	Password hashing and verification

Technology	Version	Purpose
Cloudinary	2.5.1	Cloud-based image storage and optimization
Multer	2.0.2	Multipart form data handling for file uploads
express-validator	7.2.1	Request validation middleware
express-rate-limit	7.5.1	API rate limiting for security
Helmet	7.2.0	Security headers middleware
CORS	2.8.5	Cross-Origin Resource Sharing configuration
Morgan	1.10.0	HTTP request logging
Compression	1.8.1	Response compression for performance

Development Tools

- **ESLint:** Code linting and style enforcement
- **Vitest:** Unit testing framework
- **Testing Library:** React component testing
- **Nodemon:** Auto-restart development server
- **dotenv:** Environment variable management

Infrastructure & Services

- **MongoDB Atlas:** Cloud-hosted MongoDB database
- **Cloudinary:** Image CDN and transformation service
- **Render/Vercel:** Cloud hosting platforms
- **Git:** Version control system

Core Features

1. User Management

Registration & Authentication

- Phone number and email-based registration
- Password authentication with bcrypt hashing
- JWT token-based session management
- Role-based access control (Farmer, Buyer, Admin)
- Profile management with photo upload

User Profiles

- Personal information (name, location, contact details)
- Profile picture upload and management
- WhatsApp number for direct communication
- Activity history and statistics

2. Marketplace Features

For Farmers:

- **Create Listings:** Post produce with photos, quantity, price, and location
- **Manage Listings:** Edit, update status (available/sold), or delete listings
- **Dashboard:** View all personal listings and performance metrics
- **Market Prices:** Access current market prices for informed pricing decisions

For Buyers:

- **Browse Marketplace:** View all available produce listings
- **Search & Filter:** Find specific crops by type, location, and price range
- **Contact Farmers:** Direct phone call or WhatsApp integration
- **Market Prices:** View current market rates for major crops

3. Listing Management

Listing Creation:

- Support for three major crops: Maize, Rice, and Yam
- Multiple image uploads (up to 5 images per listing)
- Quantity and unit specification
- Price per unit setting
- Location information
- Automatic farmer information attachment

Listing Display:

- Grid and card-based layout for easy browsing
- High-quality image display with Cloudinary optimization
- Farmer contact information (phone, WhatsApp)
- Status indicators (Available, Sold)
- Timestamp for listing age

4. Market Price Information

Price Display:

- Current prices for Maize, Rice, and Yam
- Price trend indicators (up, down, stable)
- Percentage change from previous price
- Last updated timestamp
- Location-specific pricing (Tamale market)

Price Management (Admin Only):

- Update market prices for all crops
- Historical price tracking
- Batch price updates
- Change preview before saving

5. Admin Dashboard

Overview Dashboard:

- Key performance metrics (users, listings, revenue, activity)
- Quick action cards for common tasks
- Recent activity feed
- Platform health indicators

User Management:

- View all registered users
- Search and filter by role, name, or contact
- Activate/deactivate user accounts
- Delete users (with cascade deletion of listings)
- User statistics and analytics

Listing Management:

- View all platform listings
- Search and filter by crop, location, status
- Delete inappropriate or outdated listings
- Listing statistics and trends

Analytics Dashboard:

- Revenue tracking and growth metrics
- User growth analysis (farmers vs buyers)
- Crop distribution visualization
- Top locations by activity
- Platform engagement metrics
- Conversion rate tracking

6. Progressive Web App (PWA)

Offline Functionality:

- Service Worker for offline caching
- Cached pages and assets
- Offline fallback pages
- Background sync for data updates

Installation:

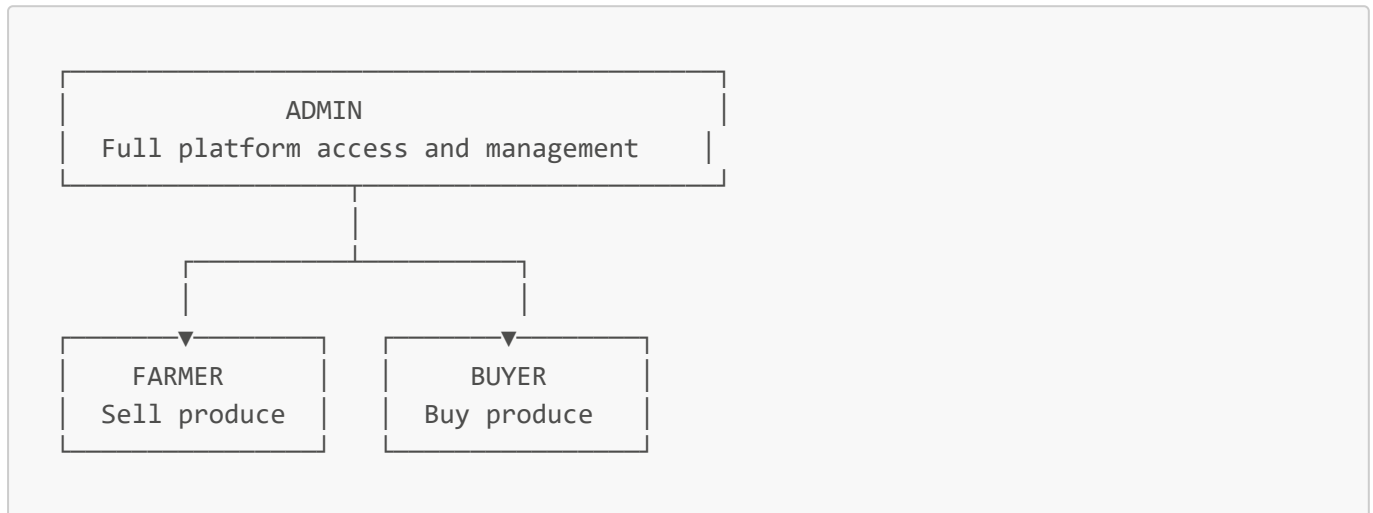
- Add to Home Screen prompt
- Native app-like experience
- Standalone display mode
- Custom splash screen

Performance:

- Fast load times with code splitting
 - Image optimization via Cloudinary
 - Lazy loading for images and components
 - Response compression
-

User Roles & Permissions

Role Hierarchy



Farmer Permissions

Can:

- Create and manage own produce listings
- Upload listing images
- Update listing status (available/sold)
- Delete own listings
- View and edit own profile
- Access market price information
- View marketplace as a buyer

Cannot:

- Access admin dashboard
- Modify other farmers' listings
- Update market prices
- Manage other users

Buyer Permissions

Can:

- Browse all available listings
- Search and filter marketplace
- View farmer contact information
- Access market price information

- View and edit own profile

Cannot:

- Create produce listings
- Access farmer dashboard
- Access admin dashboard
- Modify any listings

Admin Permissions

Can:

- Access admin dashboard
- View all users and listings
- Delete any user or listing
- Activate/deactivate user accounts
- Update market prices
- View analytics and reports
- Perform all farmer and buyer actions

Cannot:

- (No restrictions - full platform access)
-

Data Models

User Model

Purpose: Stores information about all platform users (farmers, buyers, admins)

Fields:

- **_id**: Unique identifier (MongoDB ObjectId)
- **name**: User's full name (required, max 80 characters)
- **email**: Email address (required, unique, indexed)
- **phoneNumber**: Phone number (required, unique, indexed)
- **password**: Hashed password (required, min 6 characters, not returned in queries)
- **role**: User role - "farmer", "buyer", or "admin" (required, indexed)
- **location**: User's location (required, indexed)
- **profilePicture**: URL to profile image (optional)
- **whatsappNumber**: WhatsApp contact number (optional)
- **createdAt**: Account creation timestamp (auto-generated)

Indexes:

- Email (unique)
- Phone number (unique)
- Role
- Location

Security:

- Passwords hashed with bcrypt (10 salt rounds)
- Password field excluded from queries by default
- Password comparison method for authentication

Listing Model

Purpose: Stores produce listings created by farmers

Fields:

- **_id**: Unique identifier (MongoDB ObjectId)
- **farmerId**: Reference to User model (required, indexed)
- **cropType**: Type of crop - "maize", "rice", or "yam" (required, indexed)
- **quantity**: Amount of produce (required, minimum 1)
- **unit**: Unit of measurement (default: "bag", max 20 characters)
- **price**: Price per unit (required, minimum 0, indexed)
- **location**: Produce location (required, indexed)
- **images**: Array of image URLs (max 5 images)
- **status**: Listing status - "available" or "sold" (default: "available", indexed)
- **createdAt**: Listing creation timestamp (auto-generated)
- **updatedAt**: Last update timestamp (auto-generated)

Indexes:

- Farmer ID
- Crop type
- Price
- Location
- Status
- Composite index: (cropType, location, price, status)
- Created date (descending)

Relationships:

- Belongs to User (farmer)
- Cascade delete when farmer is deleted

Market Price Model

Purpose: Stores current market prices for major crops

Fields:

- **_id**: Unique identifier (MongoDB ObjectId)
- **cropType**: Type of crop - "maize", "rice", or "yam" (required, unique, indexed)
- **currentPrice**: Current market price (required, minimum 0)
- **previousPrice**: Previous market price (default: 0, minimum 0)
- **location**: Market location (default: "Tamale", indexed)
- **unit**: Price unit (default: "100kg")

- **lastUpdated:** Last price update timestamp (default: current date)

Indexes:

- Crop type (unique)
- Location

Business Rules:

- One price record per crop type
 - Previous price automatically saved when updating
 - Admin-only updates
-

Authentication & Security

Authentication Flow

Registration Process:

1. User submits registration form (name, email, phone, password, role, location)
2. Client validates input using Zod schema
3. Server validates input using express-validator
4. Server checks for existing email/phone number
5. Server hashes password with bcrypt
6. Server creates user record in database
7. Server generates JWT token with user ID and role
8. Server returns token and user data (excluding password)
9. Client stores token in localStorage
10. Client redirects to role-appropriate dashboard

Login Process:

1. User submits login form (email/phone and password)
2. Client validates input
3. Server finds user by email or phone number
4. Server compares submitted password with hashed password
5. Server generates JWT token if password matches
6. Server returns token and user data
7. Client stores token in localStorage
8. Client redirects to role-appropriate dashboard

Protected Route Access:

1. User navigates to protected route
2. Client checks for valid token in localStorage
3. Client includes token in Authorization header
4. Server middleware extracts and verifies token
5. Server loads user from database
6. Server checks user role against required roles

7. If authorized: request proceeds to controller
8. If unauthorized: server returns 401/403 error

Security Measures

Password Security:

- Minimum 6 characters required
- Bcrypt hashing with 10 salt rounds
- Password field never returned in API responses
- Secure password comparison using bcrypt

Token Security:

- JWT tokens signed with secret key
- Tokens expire after 14 days
- Tokens include user ID, role, and phone number
- Token verification on every protected request
- Invalid tokens trigger automatic logout

API Security:

- Rate limiting on all API endpoints (100 requests per 15 minutes)
- Helmet middleware for security headers
- CORS configuration for allowed origins
- Request size limits (200KB max)
- Input validation on all endpoints
- SQL injection prevention via Mongoose
- XSS protection via input sanitization

Data Security:

- MongoDB connection over TLS
- Environment variables for sensitive data
- No sensitive data in client-side code
- Secure image uploads via Cloudinary
- File type validation for uploads
- File size limits for uploads

Authorization:

- Role-based access control (RBAC)
- Middleware checks for required roles
- Resource ownership verification
- Admin-only endpoints protected
- Cascade permissions for related resources

API Architecture

API Design Principles

1. **RESTful:** Standard HTTP methods (GET, POST, PUT, PATCH, DELETE)
2. **Resource-Based:** URLs represent resources, not actions
3. **Stateless:** No server-side session storage
4. **JSON:** All requests and responses use JSON format
5. **Versioned:** API version in URL path (/api/v1)
6. **Paginated:** Large result sets support pagination
7. **Filtered:** Query parameters for filtering and searching

API Endpoints Overview

Authentication Endpoints:

- `POST /api/auth/register` - Create new user account
- `POST /api/auth/login` - Authenticate user
- `GET /api/auth/me` - Get current user profile
- `PUT /api/auth/me` - Update current user profile

Listing Endpoints:

- `GET /api/listings` - Get all listings (with filters)
- `GET /api/listings/:id` - Get single listing
- `GET /api/listings/me` - Get current farmer's listings
- `POST /api/listings` - Create new listing (farmer only)
- `PUT /api/listings/:id` - Update listing (owner only)
- `DELETE /api/listings/:id` - Delete listing (owner only)
- `PATCH /api/listings/:id/sold` - Mark listing as sold (owner only)

Price Endpoints:

- `GET /api/prices` - Get all market prices
- `POST /api/prices` - Update market prices (admin only)

Admin Endpoints:

- `GET /api/admin/stats` - Get dashboard statistics
- `GET /api/admin/analytics` - Get detailed analytics
- `GET /api/admin/users` - Get all users
- `DELETE /api/admin/users/:id` - Delete user
- `PATCH /api/admin/users/:id/status` - Toggle user status
- `GET /api/admin/listings` - Get all listings
- `DELETE /api/admin/listings/:id` - Delete listing

Request/Response Patterns

Standard Success Response:

```
{
  "success": true,
  "data": { ... },
}
```

```
"message": "Operation successful"
}
```

Standard Error Response:

```
{
  "success": false,
  "error": "Error message",
  "details": [ ... ]
}
```

Pagination Response:

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "pages": 8
  }
}
```

Middleware Stack

Request Processing Order:

1. CORS handling
2. Security headers (Helmet)
3. Request logging (Morgan)
4. Body parsing (JSON, URL-encoded)
5. Compression
6. Rate limiting
7. Authentication (protect middleware)
8. Authorization (authorize middleware)
9. Input validation
10. Route handler
11. Error handling

Frontend Architecture

Component Structure

Layout Components:

- **FarmShell**: Main application shell with navigation
- **RequireAuth**: Protected route wrapper with role checking
- **RoleBasedRedirect**: Automatic redirection based on user role

Page Components:

- **Landing**: Public landing page
- **FarmerLogin**: Login page with role selection
- **FarmerCreateAccount**: Registration page
- **FarmerDashboard**: Farmer's main dashboard
- **BuyerMarketplace**: Buyer's marketplace view
- **AdminDashboard**: Admin overview dashboard
- **AdminUsers**: User management page
- **AdminListings**: Listing management page
- **AdminPrices**: Price management page
- **AdminAnalytics**: Analytics dashboard
- **PostProduce**: Create listing form
- **MyListings**: Farmer's listing management
- **MarketPrices**: Public market prices page
- **UserProfile**: User profile and settings
- **NotFound**: 404 error page

Shared Components:

- **ActionTile**: Dashboard action card
- **ProduceCard**: Listing display card
- **LogoutButton**: Logout functionality
- **NavLink**: Navigation link component
- **OfflineNotice**: Offline status indicator
- **InstallPWA**: PWA installation prompt

UI Components (shadcn/ui):

- 50+ accessible, customizable components
- Built on Radix UI primitives
- Styled with Tailwind CSS
- Includes: Button, Card, Dialog, Form, Input, Table, Toast, etc.

State Management

Server State (TanStack Query):

- API data fetching and caching
- Automatic background refetching
- Optimistic updates
- Error handling and retry logic
- Query invalidation

Client State:

- React hooks (useState, useEffect, useReducer)
- Context API for global state
- Local storage for persistence
- Form state with React Hook Form

Session State:

- JWT token in localStorage
- User data in localStorage
- Session utilities in lib/session.ts
- Automatic token expiration handling

Routing Strategy

Public Routes:

- / - Landing page
- /login - Login page
- /create-account - Registration page

Farmer Routes (Protected):

- /farmer - Farmer dashboard
- /post - Create listing
- /listings - Manage listings

Buyer Routes (Protected):

- /buyer - Marketplace
- /marketplace - Marketplace (alias)

Admin Routes (Protected):

- /admin - Admin dashboard
- /admin/users - User management
- /admin/listings - Listing management
- /admin/prices - Price management
- /admin/analytics - Analytics

Shared Routes (Protected):

- /profile - User profile
- /prices - Market prices

Styling Approach

Design System:

- Custom color palette (forest green, warm brown, golden yellow)
- Consistent spacing scale
- Typography hierarchy
- Component variants

- Responsive breakpoints

Tailwind Configuration:

- Custom colors and themes
- Extended spacing and sizing
- Custom animations
- Typography plugin
- Form plugin

Component Styling:

- Utility-first approach
 - Component composition
 - Variant-based styling
 - Responsive modifiers
 - Dark mode support (future)
-

Progressive Web App (PWA)

PWA Features

Installability:

- Web App Manifest for installation
- Add to Home Screen prompt
- Standalone display mode
- Custom app icons (8 sizes)
- Splash screen configuration
- App shortcuts for quick access

Offline Capability:

- Service Worker for caching
- Offline fallback pages
- Background sync for data
- Cache-first strategy for assets
- Network-first strategy for API calls

Performance:

- Code splitting for faster loads
- Lazy loading for routes and images
- Asset preloading
- Response compression
- Image optimization via Cloudinary

Native-Like Experience:

- Full-screen mode

- Native navigation
- Touch gestures
- Push notifications (future)
- Background sync (future)

Service Worker Strategy

Caching Strategy:

1. **Cache First:** Static assets (CSS, JS, images)
2. **Network First:** API calls and dynamic content
3. **Stale While Revalidate:** Frequently updated content
4. **Cache Only:** Offline fallback pages

Cache Management:

- Versioned cache names
- Automatic cache cleanup
- Cache size limits
- Selective caching based on routes

Manifest Configuration

App Identity:

- Name: "Farm Market - Connect Farmers & Buyers"
- Short Name: "Farm Market"
- Description: Marketplace for fresh produce
- Theme Color: Forest green (#2D5F2E)
- Background Color: Warm beige (#F5F3EF)

Display:

- Standalone mode (no browser UI)
- Portrait orientation
- Full scope access

Icons:

- 8 icon sizes (72px to 512px)
- PNG format
- Maskable for adaptive icons

Shortcuts:

- Farmer Dashboard
- Browse Marketplace
- Market Prices

Deployment & Infrastructure

Deployment Architecture

Frontend Deployment:

- Platform: Vercel or Netlify
- Build: Vite production build
- CDN: Global edge network
- SSL: Automatic HTTPS
- Environment: Production environment variables

Backend Deployment:

- Platform: Render or Railway
- Runtime: Node.js LTS
- Process: PM2 or native process manager
- SSL: Automatic HTTPS
- Environment: Production environment variables

Database:

- Service: MongoDB Atlas
- Tier: Shared or Dedicated cluster
- Region: Closest to application servers
- Backup: Automatic daily backups
- Monitoring: Atlas monitoring tools

Image Storage:

- Service: Cloudinary
- Plan: Free or paid tier
- Optimization: Automatic image transformation
- CDN: Global content delivery
- Backup: Cloudinary storage

Environment Configuration

Client Environment Variables:

- `VITE_API_BASE_URL`: Backend API URL

Server Environment Variables:

- `NODE_ENV`: Environment (development/production)
- `PORT`: Server port
- `MONGO_URI`: MongoDB connection string
- `JWT_SECRET`: JWT signing secret
- `JWT_EXPIRES_IN`: Token expiration time
- `CLIENT_ORIGIN`: Allowed CORS origins
- `CLOUDINARY_CLOUD_NAME`: Cloudinary account name
- `CLOUDINARY_API_KEY`: Cloudinary API key
- `CLOUDINARY_API_SECRET`: Cloudinary API secret

Monitoring & Logging

Application Monitoring:

- Server logs via Morgan
- Error tracking
- Performance metrics
- API response times

Database Monitoring:

- MongoDB Atlas monitoring
- Query performance
- Connection pool status
- Storage usage

Infrastructure Monitoring:

- Server health checks
- Uptime monitoring
- Resource usage (CPU, memory)
- Network traffic

Backup & Recovery

Database Backups:

- Automatic daily backups via MongoDB Atlas
- Point-in-time recovery
- Backup retention policy
- Manual backup capability

Image Backups:

- Cloudinary automatic storage
- Version history
- Deletion recovery window

Code Backups:

- Git version control
- GitHub repository
- Branch protection
- Release tags

Future Enhancements

Short-Term Enhancements (3-6 months)

User Features:

- SMS OTP verification for registration and login
- Email notifications for listing updates
- Saved searches and favorites
- Listing expiration and auto-renewal
- Bulk listing creation for farmers
- Advanced search with multiple filters
- Listing comparison tool

Admin Features:

- Bulk user operations
- Advanced analytics with date ranges
- Export reports (CSV, PDF)
- Activity audit logs
- Automated price updates from external sources
- User verification badges
- Content moderation tools

Technical Improvements:

- Refresh token implementation
- HTTP-only cookie authentication
- Real-time updates via WebSockets
- Push notifications
- Image compression before upload
- Pagination for all list views
- API response caching

Medium-Term Enhancements (6-12 months)**Platform Features:**

- In-app messaging between farmers and buyers
- Transaction management system
- Payment integration (Mobile Money)
- Order tracking and fulfillment
- Rating and review system
- Farmer verification program
- Buyer credit system

Mobile App:

- Native iOS app
- Native Android app
- Enhanced offline capabilities
- Camera integration for listing photos
- Location services for nearby listings
- Push notifications

Analytics:

- Predictive price analytics
- Demand forecasting
- Seasonal trend analysis
- Farmer performance insights
- Buyer behavior analytics

Long-Term Vision (12+ months)

Marketplace Expansion:

- Support for additional crops and products
- Multi-region support beyond Tamale
- International buyer access
- Wholesale marketplace
- Contract farming platform

Financial Services:

- Farmer financing and loans
- Insurance products
- Savings programs
- Payment plans for buyers

Supply Chain:

- Logistics and delivery integration
- Warehouse management
- Quality certification
- Cold storage tracking
- Transportation coordination

Community Features:

- Farmer cooperatives
- Knowledge sharing platform
- Agricultural training resources
- Weather and farming tips
- Market news and updates

AI & Automation:

- Automated price recommendations
- Crop disease detection
- Yield prediction
- Chatbot support
- Automated listing categorization

Conclusion

Farm Market Platform represents a comprehensive solution to connect farmers and buyers in the agricultural marketplace. Built with modern web technologies and following best practices in security, performance, and user experience, the platform provides a solid foundation for growth and expansion.

The mobile-first, offline-capable Progressive Web App ensures accessibility for users with varying levels of connectivity and device capabilities. The role-based architecture supports farmers, buyers, and administrators with tailored experiences and appropriate access controls.

With a clear roadmap for future enhancements, the platform is positioned to evolve into a complete agricultural marketplace ecosystem, supporting not just produce listings but also financial services, logistics, and community building.

Document End

For technical support or questions about this documentation, please contact the development team.